Simple Circular Linked List:

1. Insertion at beginning:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *next;
};
struct node *head = NULL;
void beginning() {
    struct node *first, *second, *tail;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    tail = (struct node*)malloc(sizeof(struct node));
    first->data = 100;
    second->data = 300;
    tail->data = 500;
    head = first;
    first->next = second;
    second->next = tail;
    tail->next = head;
}
void insertbeginning(int data){
    struct node *newnode, *tail;
    newnode = (struct node*)malloc(sizeof(struct node));
```

Output

```
list:50 100 300 500

=== Code Execution Successful ===
```

main.c

```c
    newnode->data = data;
    tail = head;
    while(tail->next != head){
        tail = tail->next;
    }
    newnode->next = head;
    tail->next = newnode;
    head = newnode;
}
void display(){
    struct node *ptr = head;
    if(head == NULL){
        printf("List is empty");
        return;
    }
    printf("%d ", ptr->data);
    ptr = ptr->next;
    while(ptr != head){
        printf("%d ", ptr->data);
        ptr = ptr->next;
    }
}
int main(){
```

Output

```
list:50 100 300 500

=== Code Execution Successful ===
```

```c
int main(){
    beginning();
    insertbeginning(50);
    printf("list:");
    display();
    return 0;
}
```

2. Insert at end:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 59;
    second->data = 77;
    third->data = 90;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 59 77 90 80

=== Code Execution Successful ===
```

main.c

```c
}
void insertend(){
    struct node *ptr,*newnode;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 80;
    ptr = head;
    while(ptr->next != NULL){
        ptr = ptr->next;
    }
    ptr->next = newnode;
    newnode->prev = ptr;
    newnode->next = NULL;
}
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
    beginning();
    insertend();
```

Output

```
List: 59 77 90 80

=== Code Execution Successful ===
```

```c
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
    beginning();
    insertend();
    printf("List: ");
    display();
    return 0;
}
```

3. Insertion at Position:

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 20;
    second->data = 40;
    third->data = 60;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 20 40 30 60

=== Code Execution Successful ===
```

```c
}
void insertpos(int pos){
    struct node *ptr,*newnode;
    int i;
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = 30;
    ptr = head;
    for(i=1;i<pos-1;i++){
        ptr = ptr->next;
    }
    newnode->next = ptr->next;
    newnode->prev = ptr;
    ptr->next->prev = newnode;
    ptr->next = newnode;
}
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
```

```
List: 20 40 30 60

=== Code Execution Successful ===
```

```c
int main(){
    beginning();
    insertpos(3);
    printf("List: ");
    display();
    return 0;
}
```

4. Deletion at Beginning:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 20;
    second->data = 45;
    third->data = 60;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 45 60

=== Code Execution Successful ===
```

main.c

```c
}
void deletebegin(){
    struct node *ptr;
    if(head == NULL){
        printf("List is empty\n");
        return;
    }
    ptr = head;
    head = head->next;
    if(head != NULL){
        head->prev = NULL;
    }
    free(ptr);
}
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
    beginning();
```

Output

```
List: 45 60

=== Code Execution Successful ===
```

```c
    beginning();
    deletebegin();
    printf("List: ");
    display();
    return 0;
}
```

5. Deletion at End:

main.c

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 66;
    second->data = 70;
    third->data = 89;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 66 70

=== Code Execution Successful ===
```

main.c

```c
}
void deleteend(){
    struct node *ptr;
    if(head == NULL){
        printf("List is empty\n");
        return;
    }
    ptr = head;
    while(ptr->next != NULL){
        ptr = ptr->next;
    }
    if(ptr->prev == NULL){
        head = NULL;
    }
    else{
        ptr->prev->next = NULL;
    }

    free(ptr);
}
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
```

Output

```
List: 66 70

=== Code Execution Successful ===
```

```c
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
    beginning();
    deleteend();
    printf("List: ");
    display();
    return 0;
}
```

6. Deletion at Position:

main.c

Share Run

```c
#include <stdio.h>
#include <stdlib.h>
struct node{
    int data;
    struct node *prev;
    struct node *next;
};
struct node *head = NULL;
void beginning(){
    struct node *first,*second,*third;
    first = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));
    third = (struct node*)malloc(sizeof(struct node));
    first->data = 210;
    second->data = 440;
    third->data = 600;
    head = first;
    first->prev = NULL;
    first->next = second;
    second->prev = first;
    second->next = third;
    third->prev = second;
    third->next = NULL;
```

Output

```
List: 210 600

=== Code Execution Successful ===
```

main.c

Share Run

```c
}
void deletepos(int pos){
    struct node *ptr;
    int i;
    if(head == NULL){
        printf("List is empty\n");
        return;
    }
    ptr = head;
    for(i=1; i<pos; i++){
        ptr = ptr->next;
    }
    if(ptr->prev == NULL){
        head = ptr->next;
        if(head != NULL){
            head->prev = NULL;
        }
    }
    else if(ptr->next == NULL){
        ptr->prev->next = NULL;
    }
    else{
        ptr->prev->next = ptr->next;
```

Output

```
List: 210 600

=== Code Execution Successful ===
```

```c
    else{
        ptr->prev->next = ptr->next;
        ptr->next->prev = ptr->prev;
    }
    free(ptr);
}
void display(){
    struct node *ptr = head;
    while(ptr != NULL){
        printf("%d ",ptr->data);
        ptr = ptr->next;
    }
}
int main(){
    beginning();
    deletepos(2);
    printf("List: ");
    display();
    return 0;
}
```